

《软件架构与中间件》课程知识汇总

授课教师：涂志莹 博士，哈尔滨工业大学计算机科学与技术学院，企业与服务计算研究中心

联系方式：tzy_hit@hit.edu.cn

考核方式：出勤与研讨（10%）+ 实验（30%）+ 期末考试（60%）

第0章 课程简介

课程目标

- 在理解软件工程过程、软件工程方法论和初步掌握软件开发方法的基础上，进一步学习分析复杂的软件系统开发问题的方法
- 掌握基于架构思维的复杂软件系统开发方法
- 全面提升学生的复杂工程问题分析能力、软件系统架构设计能力、典型软件系统开发及维护能力、面向性能的软件系统开发及优化能力等
- 为学生进入软件企业工作和从事软件工程领域科研工作奠定良好的技术基础

学术诚信要求

- All work must be your own
- Submitting identical assignments will be considered cheating
- Giving access to your work to another person is not allowed
- No cheating in any form will be tolerated
- 出勤要求：日常随机点名，点名3次未到者，取消考试资格

推荐教材

1. Mary Shaw, David Garlan. *Software Architecture: Perspectives on an Emerging Discipline*. 清华大学出版社, 1998
2. Len Bass. *软件架构实践 (第4版) (Software Architecture in Practice, 4th Ed)*. 2023
3. Till Adam. *架构之美*. 机械工业出版社, 2009
4. Alex Homer, et al. *云计算架构设计模式*. 华中科技大学出版社, 2017
5. 赖永炫. *中间件技术基础与Java实践*. 清华大学出版社, 2024
6. 李伟. *RocketMQ分布式消息中间件：核心原理与最佳实践*. 电子工业出版社, 2023
7. 叶文柯. *Spring技术内幕：深入解析Spring架构与设计原理*. 机械工业出版社, 2012
8. 倪超. *从PAXOS到ZOOKEEPER——分布式一致性原理与实践*. 电子工业出版社, 2015
9. Josiah L. Carlson. *Redis实战*. 人民邮电出版社, 2015
10. 周志明. *凤凰架构：构建可靠的大型分布式系统*. 机械工业出版社, 2023

第1章 系统体系结构的内涵及要点

1.1 架构的起源与定义

"架构(Architecture)"一词起源于建筑学，核心问题是：如何使用基本的建筑模块构造一座完整的建筑？包含两个因素：

- 基本的建筑模块：砖、瓦、灰、沙、石、预制梁、柱、屋面板
- 建筑模块之间的粘接关系：如何把这些模块有机组合起来形成整体建筑
- 建筑设计原则：**坚固、实用、美观**——建筑设计不仅是一门科学，而且是一项艺术

计算机体系结构类比：如何将设备组装起来形成完整的计算机硬件系统？

- 基本硬件模块：控制器、运算器、内存存储器、外存储器、输入设备、输出设备
- 硬件模块之间的连接关系：总线
- 计算机架构风格：以存储程序原理为基础的冯·诺依曼结构、存储系统的层次结构、并行处理机结构、流水线结构、多核CPU

架构的共性要素：

- 一组基本的构成要素——**构件**
- 这些要素之间的连接关系——**连接件**
- 要素连接后的拓扑结构——**物理分布**
- 作用于要素或连接关系上的限制条件——**约束**
- **质量——性能**

1.2 软件架构(SA)的四种经典定义

定义来源	公式	核心要点
早期定义	SA = {components, connectors, constraints}	构件、连接件、约束
IEEE 1471	Architecture = {component, connector, environment, principle}	构件、构件间关系、构件与环境关系、设计与演化原理
Perry & Wolf	SA = {elements, form, rational}	处理元素、数据元素、连接元素；专有属性和关系；准则
Bass等 (SAP)	SA = {elements, relations, properties}	用于推理系统的结构集合，包含软件元素、关系及两者的属性

关于SA定义的共识： "There is no standard, universally-accepted definition of the term, for software architecture is a field in its infancy, although its roots run deep in software engineering."

SA关注的Structural issues：

- Organization of the system as a composition of components
- Global control structures
- Protocols for communication, synchronization, and data access

- Assignment of functionality to design elements
- Composition of design elements
- Physical distribution
- Scaling and performance
- Dimensions of evolution
- Selection among design alternatives

软件架构的综合定义：提供了一个结构、行为和属性的高级抽象，从一个较高的层次来考虑组成系统的构件、构件之间的连接，以及由构件与构件交互形成的拓扑结构。这些要素应该满足一定的限制，遵循一定的设计规则，能够在一定的环境下进行演化。反映系统开发中具有重要影响的设计决策，便于各种人员的交流，反映多种关注。

核心公式：Architecture = Components + Connectors + Topology + Constraints + Performance

1.3 为什么需要软件架构

Edsger Dijkstra(1930-2002)最早指出SA的重要性: "..the larger the project, the more essential the structuring!" (1968)

软件规模增长的现实：

- Windows 95: 1500万行 → Windows 10: 5000万行
- 大型ERP软件：功能模块>1000个、数据表>1000张、用户>1000人
- 阿里巴巴2018双11：2分05秒成交额破100亿，全天交易额2135亿元，全天实时计算处理峰值每秒17.18亿条

软件开发范式演化：

年代	范式	核心公式
1960's	面向过程	程序 = 算法 + 数据结构
1970's	模块化	程序 = 子程序 + 子程序
1980's	面向对象	程序 = 对象 + 对象
1990's	基于构件	程序 = 构件 + 连接件
2000's	面向服务	系统 = 服务 + 服务总线
now	云计算	系统 = 服务集群 + 中间件 (Cloud Computing, Programming-distributed-in-the-world)

关键结论：对于大规模的复杂软件系统来说，对系统全局结构的设计比起对算法的选择和数据结构的设计明显重要得多。很多质量需求主要体现在架构中而不是功能模块内部的实现中。

1.4 软件架构的作用

三大作用：

- 1. **交流的手段(Communication)**: It is a vehicle for communication among stakeholders. 在软件设计者、最终用户之间方便交流
- 2. **可复用的模型(Reuse)**: It is a reusable, transferable abstraction of a system. 可重复利用的、可转移的系统抽象
 - Can be applied to other systems exhibiting similar requirements (用到其它的项目)
 - Can promote large-scale reuse and software product lines (提高大规模重复利用率)
- 3. **关键决策的体现(Design Decisions)**: It is the representation of the earliest design decisions that has the most significant influence on system qualities. It shows the tradeoffs:
 - Between performance and security (性能与安全性)
 - Between maintainability and reliability (可维护性与可靠性)
 - Between the cost of the current development effort and the cost of future developments (当前开发费用和未来开发代价)

SA在软件生命周期各阶段的价值：

- 项目规划：考虑项目的规模、复杂度、可行性
- 需求分析：利用SA支持用户、项目负责人、系统架构师、程序员、测试人员之间交流和协商
- 软件设计：从不同视角审查备选的SA，找出合理的平衡方案；参考经典SA风格，设计系统架构模型
- 软件实现：各开发团队按照SA规定的"构件及其之间的相互关系"进行开发
- 测试与评审：根据SA的约束条件，对软件的质量属性进行测试
- 维护与升级：把SA文档作为维护和升级的重要依据

SA设计的特点：

- 起点模糊：用户需求尚没有明确定义时就要进行SA设计——故而需要交流与反复
- 高抽象层次：SA处理的是高层次的系统构件或子系统之间的关系，而非变量、函数等低层次概念
- 分布决策：来自用户、架构师、测试人员等多方面
- 贯穿全局：SA设计活动在项目开始时进行，但其作用贯穿整个项目周期

1.5 架构设计的四种观点

观点	核心理念
全局观(Overall View)	从全局角度对软件进行设计的"视图"，而非仅仅关注底层算法细节
折中观(Tradeoff View)	用户提出的各类功能与非功能需求之间存在大量"矛盾"，必须权衡
交流观(Communication View)	参与软件研发的各成员需要公共"媒介"进行沟通
复用观(Reuse View)	通过利用已存在的"软件资产"，提高效率、降低成本

经典案例——圣·玛利亚百花大教堂：

- 1296年，佛罗伦萨开始动工修建，但直到1418年仍未完工
- WHY? 没人提供教堂巨大拱顶和圆顶以及起重设备的设计方案
- 建筑师布鲁内莱斯基(1377-1446)被任命为总建筑师，建成意大利文艺复兴式建筑的第一个作品
- 挑战：在没有钢筋、混凝土等现代建材的时代，如何筑成直径43.7米的石砌砖造圆顶？如何把37000吨大理石安稳放置在50米高空中？

1.6 软件质量属性与架构的关系

软件质量Software Qualities：软件与明确的和隐含的需求相一致的程度。注意：如果提高一个质量，经常会影响（提高或降低）其他质量。

主要质量属性及架构师应对手段：

质量属性	说明	架构师手段
性能(Performance)	软件的"时间-空间"效率	集群、缓存、异步
安全性(Security)	阻止未授权用户的使用企图	防火墙、入侵检测、加密、单一入口
易用性(Usability)	用户使用软件的容易程度	统一风格界面、定制
重用性(Reusability)	减少代码编写，提高开发效率	构件Component、框架Framework、软件产品线Product Line
健壮性/可用性 (Robustness/Availability)	异常情况下软件正常运行的能力(容错、恢复)	集群、灾备、故障转移
可修改性(Modifiability)	软件适应"变化"的能力	稳定的功能分解、设计模式、分层、解耦
可测试性(Testability)	软件是否易于被测试	一致的错误处理方式
集成性(Integrability)	让分别开发的组件在一起正确工作	最小化接口复杂度、统一命名规范、遵循标准
移植性(Portability)	不经修改或稍加修改就可运行于不同软硬件环境	开发语言选择、分层、遵循标准
兼容性(Compatibility)	不同产品相互交换信息的能力	API设计、遵循标准
经济性(Economy)	开发成本、时间和对市场的适应能力	重用性、可修改性
正确性(Correctness)	软件按照需求正确执行任务的能力	—
完备性(Completeness)	软件能够支持用户所需求的全部功能	—

关键认识：

- 架构的选择极大地影响部分软件质量，但不是全部

- 架构只为获得某个质量创造条件，但并不能保证肯定获得
- 我们能做的是使架构：可文档化、易于理解、可度量、可复用、易于交流和执行

1.7 软件架构师

对于大型复杂软件系统，需要专业的架构师来负责系统架构的设计。

IBM对IT架构师的定义： Like a building architect, much of the IT Architect's work is focused on the front end of the solution life-cycle. The IT Architect may also be involved during the construction of a solution as an advocate for the client, as the ultimate authority on the architecture that was produced to address the clients business problem, and to provide technical leadership and guidance to the construction team.

架构师的素质要求： 是技术领导(可能不是一个人)、了解开发流程、精通业务领域知识、精于设计、是编程高手、优秀的表达能力、能够果断决策、是谈判专家

架构师的成长： 架构师不是教出来的，架构师是实践出来的。推荐资源：InfoQ、51CTO、ZDNet、技术大牛博客、开源项目网站等。

1.8 系统架构的演化路径

架构风格演化：

功能函数 → 对象 → 构件 → 框架 → 设计模式 → 服务
(细粒度 → 粗粒度)
面向过程 → 面向对象 → 基于构件 → 面向服务(SOA) → 云计算/边缘云/弹性云/容器云
IT技术 → 商务过程
封闭 → 开放
个人 → 企业内 → 企业间 → 全球

具体演化步骤：

1. 单一服务器架构
2. 应用服务和数据服务分离
3. 缓存的使用(Memcached等)
4. 集群与并发
5. 读写分离
6. CDN与反向代理
7. 分布式数据库与文件系统、NoSQL
8. 垂直划分业务服务
9. 分布式服务架构

1.9 中间件基础

产生背景： 计算设备之间需要通过网络进行通信，应用程序从直接基于系统软件和网络软件，演变为通过网络中间件来通信。

标志性事件：

- 1968年，IBM发布CICS交易事务控制系统，使应用软件与系统服务分离，被看作中间件技术萌芽

- 1990年，BELL实验室Tuxedo系统解决分布式交易事务控制问题，中间件正式成型
- 1994年，IBM发布消息队列服务MQ系列，解决分布式系统异步、可靠、传输的通讯服务问题，消息中间件诞生
- J2EE发布后，Java从编程语言演变为网络应用架构，成为应用服务平台的事实标准，应用服务器中间件成为中间件技术的集大成者

中间件的四个角色比喻：

1. **通道**：保障应用信息的可靠传递
2. **桥梁**：达成商务构件的互联互通
3. **框架**：促成企业应用的完整集成
4. **平台**：实现分布式应用的顺利部署

中间件的本质特征：

- 中间件是软件
- 中间件在应用软件和系统软件之间
- 中间件是一类软件而不是一种
- 中间件不是开发工具，通常包含开发、管理和部署

中间件的三大作用：

1. **屏蔽异构性**：异构性表现在计算机的软硬件差异（CPU和指令集、硬件结构、驱动程序、操作系统API、数据库存储和访问格式）。高级语言依赖于特定的编译器和OS API，它们是不兼容的
2. **实现互操作**：因为异构性，各种不同软件之间在不同平台之间不能移植或移植困难，且不能有效相互集成
3. **共性凝练和复用**：通过中间件提供简单、一致、集成的开发和运行环境，简化分布式系统的设计、编程和管理

中间件的价值：缩短开发周期、节约开发成本、降低运行成本、降低故障率、改善决策、应用系统群集/集成、减少软件维护、提高质量、改进技术、提高产品吸引力

中间件分类：

1. **应用服务类中间件**：为应用系统提供综合的计算环境和支撑平台，包括对象请求代理(ORB)中间件、事务监控交易中间件、Java应用服务器中间件
2. **应用集成类中间件**：提供各种不同网络应用系统之间的消息通信、服务集成和数据集成功能，包括消息中间件、企业集成EAI、企业服务总线(ESB)以及适配器
3. **业务架构类中间件**：将业务共性抽象至中间件，形成应用模式，如业务流程、业务模型、业务规则、交互应用

按功能细分：数据访问、消息、交易、分布式对象、分布式构件(应用服务器、DCOM)、工作流、应用集成、门户等

常见中间件实例：SQL*NET、RPC、HTTP、CORBA、J2EE、.NET、Mail System、Lotus Notes、WeChat

1.10 4+1视图模型

4+1视图模型是一种已经被标准化(UML, RUP)的方法，用来描述和文档化软件系统的体系结构。每个视图从不同的角度刻画了系统的结构属性，将它们综合在一起就形成了系统的整体架构。

视图	描述	主要UML图形	结构类型
用例视图	描述系统的典型场景与功能	Use Case Diagram	—
逻辑视图	描述系统的抽象概念与功能(类、对象、接口、模式)	Class Diagram, Communication Diagram, Sequence Diagram	静态结构
开发视图	描述子系统、模块、文件、资源及其关系	Component Diagram, Package Diagram	静态结构
进程视图	描述进程及其之间的通信协作关系	Activity Diagram, Sequence Diagram	动态结构
物理视图	描述系统如何被安装、部署与配置在分布式物理环境下	Deployment Diagram	动态结构

对于不同类型的软件系统来说，侧重的角度也有所不同。

1.11 架构设计案例：在线书店系统

功能需求：

- 用户管理：用户注册、用户验证、删除用户、用户信息修改
- 存货管理：订书、到货接收、入库、修改库存、查询
- 购物管理：购物、查看购物车、支付
- 订单实现：处理订单、打包管理、配送、查询

非功能需求：可扩展、可靠性、安全性、安全与法规

作业要求：根据身边的实际软件需求问题设计它的软件架构，使用UML建模4+1视图，提供文档化报告。

第2章 软件设计模式基础

2.1 软件设计模式的定义与意义

软件设计模式(Software Design Pattern)是面向对象软件开发人员的试验和错误的经验总结，是软件开发过程中一般问题的解决方案，也是代码编制工程化（软件工程的基石）的重要体现。

设计模式的三大价值：

- **重用代码**：避免重复造轮子
- **让代码更容易被他人理解**：提高代码的可读性
- **保证代码可靠性**：经过验证的解决方案更可靠

每种设计模式都描述了一个在日常生产、生活中不断重复发生的问题，以及该问题的核心解决方案，都有相应的原理。

开发人员的共同平台：软件设计模式提供标准的术语系统，且具体到特定情景。例如，单例设计模式意味着使用单个对象，熟悉该模式的开发人员都能理解，并可以通过这种方式告诉对方程序使用的是单例模式。

最佳的实践：软件设计模式提供了软件开发过程中面临的一般问题的最佳解决方案，有助于经验不足的开发人员通过简单快捷的方式来学习软件设计。

2.2 设计模式的三大分类（GoF 23种 + J2EE）

GoF(Gang of Four)经典著作《可复用的面向对象软件元素》(Elements of Reusable Object-Oriented Software)总结了25种模式，分为三大类。

创建型模式（Creational Patterns）—— 5种

一种在创建对象的同时**隐藏创建逻辑**的方式，而不是使用new运算符直接实例化对象。使程序在判断针对某个给定实例需要创建哪些对象时更加灵活。

1. 工厂模式(Factory Pattern)
2. 抽象工厂模式(Abstract Factory Pattern)
3. 单例模式(Singleton Pattern)
4. 建造者模式(Builder Pattern)
5. 原型模式(Prototype Pattern)

结构型模式（Structural Patterns）—— 8种

关注**类和对象的组合**。继承的概念被用来组合接口和定义组合对象获得新功能的方式。

1. 适配器模式(Adapter Pattern)
2. 桥接模式(Bridge Pattern)
3. 过滤器模式(Filter/Criteria Pattern)
4. 组合模式(Composite Pattern)
5. 装饰器模式(Decorator Pattern)
6. 外观模式(Facade Pattern)
7. 享元模式(Flyweight Pattern)
8. 代理模式(Proxy Pattern)

行为型模式（Behavioral Patterns）—— 12种

关注**对象之间的通信**。

1. 责任链模式(Chain of Responsibility Pattern)
2. 命令模式(Command Pattern)
3. 解释器模式(Interpreter Pattern)
4. 迭代器模式(Iterator Pattern)
5. 中介者模式(Mediator Pattern)
6. 备忘录模式(Memento Pattern)
7. 观察者模式(Observer Pattern)
8. 状态模式(State Pattern)
9. 空对象模式(Null Object Pattern)

10. 策略模式(Strategy Pattern)
11. 模板模式(Template Pattern)
12. 访问者模式(Visitor Pattern)

J2EE设计模式 —— 8种 (关注表示层, Sun Java Center鉴定)

1. MVC模式(MVC Pattern)
2. 业务代表模式(Business Delegate Pattern)
3. 组合实体模式(Composite Entity Pattern)
4. 数据访问对象模式(Data Access Object Pattern)
5. 前端控制器模式(Front Controller Pattern)
6. 拦截过滤器模式(Intercepting Filter Pattern)
7. 服务定位器模式(Service Locator Pattern)
8. 传输对象模式(Transfer Object Pattern)

2.3 六大设计原则

原则一：开闭原则(Open Close Principle)

- **核心思想**：对扩展开放，对修改关闭
- **具体含义**：在程序需要进行拓展的时候，不能去修改原有的代码，实现一个**热插拔**的效果。使程序的扩展性好，易于维护和升级
- **实现方式**：需要使用**接口和抽象类**

原则二：里氏代换原则(Liskov Substitution Principle)

- **核心思想**：任何基类可以出现的地方，子类一定可以出现
- **重要地位**：LSP是**继承复用的基石**。只有当派生类可以替换掉基类，且软件单位的功能不受到影响时，基类才能真正被复用
- **与OCP的关系**：是对开闭原则的补充。实现开闭原则的关键步骤是抽象化，基类与子类的继承关系是抽象化的具体实现，LSP是对实现抽象化的具体步骤的规范

原则三：依赖倒转原则(Dependence Inversion Principle)

- **重要地位**：是开闭原则的基础
- **具体内容**：针对接口编程，依赖于抽象而不依赖于具体

原则四：接口隔离原则(Interface Segregation Principle)

- **核心思想**：使用**多个隔离的接口**，比使用单个接口要好
- **具体要求**：客户端不应该依赖它不需要的接口。降低类之间的耦合度。建立单一接口，尽量细化接口，接口中的方法尽量少
- **适度原则**：一定要适度，过大会增加耦合性，过小会增加复杂性和开发成本

原则五：迪米特法则/最少知道原则(Demeter Principle)

- **核心思想**：一个实体应当尽量少地与其他实体之间发生相互作用，使得系统功能模块相对独立
- **目的**：降低系统的耦合度，使类与类之间保持**松耦合**状态

代码示例——错误示范（违反迪米特法则）：

```
public class Computer {
    public void saveCurrentTask(){ /* do something */ }
    public void closeService(){ /* do something */ }
    public void closeScreen(){ /* do something */ }
    public void closePower(){ /* do something */ }
    public void close(){
        saveCurrentTask(); closeService(); closeScreen(); closePower();
    }
}

public class Person {
    private Computer computer;
    public void clickCloseButton(){
        // 所有方法都是公开的，不知道如何调用
        computer.saveCurrentTask(); computer.closePower(); computer.close();
        // 或者 computer.closePower();
        // 或者 computer.close(); computer.closePower();
    }
}
```

代码示例——正确示范（遵循迪米特法则）：

```
public class Person {
    private Computer computer;
    public void clickCloseButton(){
        computer.close(); // 只调用一个方法
    }
}
```

原则六：合成复用原则(Composite Reuse Principle)

- **核心思想**：尽量使用**合成/聚合**的方式，而不是使用继承
- **继承优点**：容易实现并且容易修改和扩展继承来的内容
- **继承缺点**：增加了类之间的依赖，属于“**白箱**”**复用**，父类对子类透明，破坏封装性
- **合成缺点**：系统中会存在较多对象需要管理
- **重要提示**：不是不使用继承，而是尽量**先考虑合成，再考虑继承**

三种复用方式代码示例：

(1) 简单依赖（方法参数级别的临时关联）：

```
class A { public void methodA(){ System.out.println("A类的方法执行了。"); } }
class B { public void methodB(A a){ System.out.println("B类的方法执行了。"); a.methodA(); } }
```

(2) 聚合（通过setter注入，A可独立存在）：

```
class B {
    private A a;
    public A getA(){ return a; }
    public void setA(A a){ this.a = a; }
    public void methodB(){ this.a.methodA(); }
}
```

(3) 组合（B直接创建A实例，A生命周期由B管理）：

```
class B {
    private A a = new A();
    public void methodB(){ this.a.methodA(); }
}
```

2.4 单例模式(Singleton Pattern)

核心要点：确保一个类只有一个实例，提供全局访问点。

三个要点：

- 1. 单例类只能有一个实例
- 2. 单例类必须自己创建自己的唯一实例
- 3. 单例类必须给所有其他对象提供这一实例

五种实现方式：

方式	Lazy初始化	线程安全	特点
懒汉式(基础)	是	否	严格说不算单例模式
懒汉式(安全)	是	是(synchronized)	效率很低，99%情况下不需要同步
饿汉式	否	是(classloader)	无加锁，效率高，但可能浪费内存
双检锁(DCL)	是	是(双锁)	JDK1.5+，高性能
静态内部类	是	是(classloader)	达到双检锁功效

优点：减少内存开销(单实例)、避免资源多重占用

缺点：没有接口不能继承，与单一职责原则冲突

应用场景：唯一序列号生成、Web计数器、I/O与数据库连接

2.5 工厂模式(Factory Pattern)

核心要点：定义创建对象的接口，让子类决定实例化哪个类。创建过程延迟到子类进行。创建对象时不对客户端暴露创建逻辑，通过共同接口指向新创建对象。

生活类比：需要一辆汽车，直接从工厂提货，不用管汽车是怎么做出来的。

优点：知道名称即可创建、扩展性高(增加产品只需扩展工厂类)、屏蔽产品实现

缺点：每增加产品需增加具体类和工厂，系统类个数成倍增加

应用场景：

1. 日志记录器（本地硬盘/系统事件/远程服务器）
2. 数据库访问（不确定最终使用哪类数据库）
3. 连接服务器框架（需支持POP3/IMAP/HTTP三个协议）

注意事项：复杂对象适合使用工厂模式，简单对象(new即可完成)无需使用，否则会增加系统复杂度。

2.6 抽象工厂模式(Abstract Factory Pattern)

核心要点：围绕**超级工厂**创建其他工厂（工厂的工厂）。提供一个创建一系列相关或相互依赖对象的接口，无需指定具体类。

关键代码：在一个工厂里聚合多个同类产品。接口负责创建相关对象的工厂。

优点：保证客户端始终只使用同一个产品族中的对象

缺点：产品族扩展非常困难（既要在抽象Creator加代码，又要在具体实现加代码）

应用场景：QQ换皮肤（一整套一起换）、生成不同操作系统的程序

注意事项：产品族难扩展，产品等级易扩展

2.7 适配器模式(Adapter Pattern)

核心要点：将不兼容接口转换为兼容接口（作为两个不兼容接口之间的桥梁）。

生活类比：读卡器是内存卡和笔记本之间的适配器。

关键代码：适配器继承或依赖已有对象，实现目标接口。

解决方案：继承或依赖（推荐依赖）。

应用实例：

- 美国电器110V转中国220V
- JDK 1.1 Enumeration接口转1.2 Iterator接口
- JAVA JDBC

角色结构：目标接口(Target) → 适配器(Adapter, 继承或引用适配者) → 适配者(Adaptee)

类适配器 vs 双向适配器：可扩展为双向适配器，既可以把适配者接口转成目标接口，也可以反向转换。

优点：让无关联的类一起运行、提高复用、增加透明度、灵活性好

缺点：过多使用导致系统零乱。如果非必要，直接重构系统更好

使用场景：有动机地修改正常运行的系统的接口

注意事项：适配器不是在详细设计时添加的，而是解决正在服役的项目的问题

2.8 桥接模式(Bridge Pattern)

核心要点：将抽象部分与实现部分分离，使它们都可以独立变化。通过桥接接口使实体类功能独立于接口实现类。

解决问题：多种可能变化时用继承会造成**类爆炸**问题

角色结构：抽象化(Abstraction) → 扩展抽象化(Refined Abstraction) → 实现化(Implementor) → 具体实现化(Concrete Implementor)

优点：抽象和实现分离、优秀扩展能力、实现细节对客户透明

缺点：增加系统理解与设计难度

应用场景：类存在两个独立变化的维度且都需扩展；不希望用继承或多层次继承导致类急剧增加

联合使用：可与适配器模式联合。当桥接模式的实现化角色接口与现有类接口不一致时，在二者中间定义适配器连接。

2.9 代理模式(Proxy Pattern)

核心要点：为对象提供代理以控制访问。访问对象不适合或不能直接引用目标对象时，代理作为中介。也叫做**委托模式**。

解决方案：增加中间层。

关键代码：实现与被代理类组合。

静态代理：代理对象与目标对象实现同样接口。优点是不修改目标对象前提下扩展功能；缺点是代理类多，接口变更时需维护。

JDK动态代理：通过实现InvocationHandler来实现，真正代理对象由JDK运行时动态创建。

- 大大减少开发任务，降低对业务接口的依赖
- 限制：只能支持实现了Interface的类
- 重要应用：**Spring AOP的基础**

优点：中介和保护作用、可扩展目标功能（加入缓存/日志而不打开封装好的类）、降低耦合度

缺点：请求处理速度变慢、增加系统复杂度

按职责划分的8种代理：远程代理、虚拟代理、Copy-on-Write代理、保护代理、Cache代理、防火墙代理、同步化代理、智能引用代理

2.10 中介者模式(Mediator Pattern)

核心要点：定义中介对象封装对象间交互，使原有对象间耦合松散。是**迪米特法则的典型应用**。又叫调停模式。

解决问题：对象间存在大量关联关系，形成**网状结构**。

解决方案：将**网状结构**分离为**星型结构**。

角色结构：抽象中介者(Mediator) → 具体中介者(ConcreteMediator，用List管理同事对象) → 抽象同事类(Colleague，保存中介者对象) → 具体同事类(Concrete Colleague)

优化：中介者模式+单例模式 → 不定义中介者接口，将具体中介者实现为单例。同事对象不持有中介者，需要时直接获取并调用。

优点：降低类复杂度(一对多变为一对一)、各类型解耦、符合迪米特原则

缺点：中介者会庞大、变得复杂难以维护

2.11 观察者模式(Observer Pattern)

核心要点：定义对象间一对多依赖关系，状态改变时所有依赖对象自动更新。又称**发布-订阅/模型-视图模式**。

关键代码：抽象类里有ArrayList存放观察者们。

角色结构：抽象目标(Subject，管理观察者) → 具体目标(Concrete Subject，通知观察者) → 抽象观察者(Observer，包含更新方法) → 具体观察者(Concrete Observer)

重要设计原则：具体目标对象和具体观察者对象之间不能直接调用，否则将使两者紧密耦合。

Java内置支持：

- `Observable` 类：用Vector存储观察者，3个方法(addObserver/notifyObservers/setChange)
- `Observer` 接口：`void update(Observable o, Object arg)`

优点：观察者和被观察者抽象耦合、建立触发机制

缺点：通知所有观察者花费时间、可能有循环依赖导致系统崩溃、不知道目标对象怎么变化

使用场景：抽象模型两方面解耦、触发链创建 (A→B→C)

注意事项：避免循环引用、一般采用**异步方式**防止某一观察者错误导致系统卡壳

2.12 访问者模式(Visitor Pattern)

核心要点：将数据操作与数据结构分离。在不改变数据结构的前提下添加新操作。是行为类模式中**最复杂**的模式。

角色结构：抽象访问者(Visitor, visit操作) → 具体访问者(ConcreteVisitor) → 抽象元素(Element, accept操作) → 具体元素(ConcreteElement, visitor.visit(this)) → 对象结构(Object Structure, List/Set/Map容器)

优点：符合单一职责原则、优秀扩展性、灵活性

缺点：违反迪米特原则(具体元素向访问者公布细节)、违反依赖倒置原则(依赖具体类)、具体元素变更困难

应用场景：对象结构相对稳定但经常需要定义新操作；需要对对象进行多种不相关操作

第3章 计算层的软件架构技术

3.1 计算模式的变革

演进路径：

1. **单机任务**（手机、车载终端、摄像头、智能终端）
 - 想快点 → 升级处理器、并行计算
 - 想脱离客户机/多用户 → 服务器、云识别服务
2. **高性能集群**（用户规模激增至100万）
 - 集群 + 消息队列 + 负载调度
3. **高可用分布式**（用户规模爆炸至1亿+）
 - 多机房 + 冗余 + 异地多活 + 降级

单机计算模式：

- 串行计算：经典指令执行模型（读→算→写）
- 摩尔定律的“高与胖”：处理器的演进
- 多核/众核计算：从Pentium时代的单线程速度优化，演进到现代芯片平衡单线程和多线程性能。路线分为“优化速度”和“优化性能/功耗比”。架构包含本地缓存、流线化IA核心、共享缓存。

分布式计算模式：业务需求爆炸式增长 + 技术演进导致系统异构化 + 业务与技术沟通鸿沟。

- **传统垂直架构改造核心：**对应用进行服务化，使用分布式服务框架
- **依旧存在的问题：**开发维护成本高、部署效率低、代码复用难、难以适应敏捷持续交付
- **架构示意：**集群服务器上多个业务实例（商品、订单、库存）通过负载均衡器分发

大规模系统架构设计一般原则：尽可能地拆分，以达到更好的独立扩展与伸缩、更灵活的部署、更好的隔离和容错、更高的开发效率。

垂直拆分示例：

- 订单域服务独立集群（订单管理、订单稽查）
- 产品域服务独立集群（新增产品、产品查询）
- 客户域服务独立集群（客户管理、历史查询）
- 各域通过分布式服务框架连接

分布式服务框架（虚拟总线）：将核心的、公共的业务拆分出来，消费者通过标准契约消费服务，服务提供者独立打包、部署和演进，与消费者解耦。

服务治理：随着业务拆分服务数增多，需有效管控服务：

1. **生命周期管理：**规范化上线审批、测试发布流程、下线通知
2. **服务容量规划：**采集调用性能、时延、成功率、资源占用，分析容量瓶颈
3. **运行时治理：**SLA降级、在线调大超时时间、本地降级逻辑
4. **服务安全：**身份验证与权限验证

3.2 非功能度量指标

基本性能指标：

指标	定义
CPU速度	MIPS (Million Instructions Per Second)
网络带宽	Mbps (Megabits per second)
吞吐量	MIPS、TFLOPS、TPS、QPS — 系统在单位时间内处理请求的数量
RT响应时间	系统对请求作出响应的的时间
网络延时	—
并发用户数	系统可同时承载的正常使用系统功能的用户数

TPS vs QPS：

- **TPS(Transactions Per Second)：**客户机发请求时开始计时→收到响应结束计时。包含3个过程：用户请求→服务器内部处理→服务器返回。每秒完成N个全过程即TPS=N
- **QPS(Queries Per Second)：**每秒处理查询数目。一次页面访问形成一个TPS，但可能产生多次服务器请求，每次都计入QPS

可扩展性(Scalability)：系统适应不断增长的处理任务的能力

- **垂直扩展(Scale Up)：**向单节点添加资源（增加CPU或内存）
- **水平扩展(Scale Out)：**添加更多节点进行分布式计算

可用性(Availability)：系统在几乎任何时刻都可被正常访问，通常量化为一年中正常运行时间比。

系统可用性公式：可用性 = MTTF / (MTTF + MTTR)

- MTTF (Mean Time To Failure): 平均故障时间
- MTTR (Mean Time to Repair): 平均修复时间
- MTBF (Mean Time Between Failures) = MTTF + MTTR: 平均故障间隔时间

重大宕机事件:

- 2010年淘宝双11: 2100万人参与, 被迫停掉"确认收货"业务
- 2012年12306: 日均点击超10亿次, 一度无法登录
- eBay 1999年22小时宕机: 股价5天跌26%, 第二季营收少500万美元
- Google、PayPal、Facebook、新浪微博等也曾发生长时间宕机

3.3 分布式编程模型——MapReduce

背景: "摩尔定律"CPU性能约每18个月翻番, 但从2005年逐渐失效。数据量快速增加, 须借助分布式并行编程。谷歌最先提出MapReduce, Hadoop是其开源实现。

MapReduce vs 传统并行框架(如MPI):

维度	传统并行框架	MapReduce
集群架构/容错性	共享式(共享内存/存储), 容错性差	非共享式, 容错性好
硬件/价格/扩展性	刀片服务器、高速网、SAN, 贵, 扩展差	普通PC机, 便宜, 扩展好
编程/学习难度	what-how, 难	what, 简单
适用场景	实时、细粒度、计算密集型	批处理、非实时、数据密集型

分布式文件系统: 文件分布存储到多个计算机节点上, 由普通硬件构成(降低成本)。节点分为**主节点(NameNode)**和**从节点(DataNode)**。

核心思想 (数黑桃牌类比): 给每个人分配牌→每个人数黑桃→汇报数目→汇总结果。(这就是Map和Reduce)

Map和Reduce函数:

函数	输入	输出	说明
Map	<k1,v1> 如<行号,"a b c">	List(<k2,v2>) 如<"a",1> <"b",1> <"c",1>	将小数据集解析成key-value对; 每个输入<k1,v1>输出一批中间结果<k2,v2>
Reduce	<k2,List(v2)> 如<"a", <1,1,1>>	<k3,v3> 如<"a",3>	List(v2)是属于同一个k2的一批value

- MapReduce采用"分而治之"策略, 数据集切分为独立分片(split), 多Map任务并行处理
- 设计理念: **"计算向数据靠拢"**, 避免大量网络传输开销
- 框架架构: Master/Slave, Master上运行JobTracker, Slave上运行TaskTracker
- Hadoop框架用Java实现, 但MapReduce应用不一定要用Java写

四大组成部分:

1. **Client:** 提交MapReduce程序、查看运行状态

2. **JobTracker**: 资源监控和作业调度; 监控所有TaskTracker与Job健康; 跟踪任务进度和资源使用; 调度器在资源空闲时分配任务
3. **TaskTracker**: 周期性"心跳"汇报资源使用和任务进度; 用"slot"划分节点资源(CPU/内存等); slot分为Map slot和Reduce slot
4. **Task**: Map Task和Reduce Task, 由TaskTracker启动

通信特性: 不同Map任务间不通信; 不同Reduce任务间不交换信息; 用户不能显式跨机发消息; 所有数据交换通过框架自身实现。

Split概念: split是逻辑概念(数据起始位置、长度、所在节点等元数据), 划分方法由用户决定。HDFS以固定大小block存储。

任务数量设定:

- Map任务数: 每个split创建1个Map任务。理想分片大小=1个HDFS块(默认64MB)
- Reduce任务数: 依赖于可用reduce slot数目。通常设置比slot数稍少以预留资源

Shuffle过程——Map端:

- 每个Map分配缓存(默认100MB) → 溢写比例0.8
- 分区: 默认哈希函数
- 排序: 默认操作
- 合并(Combine): 不能改变最终结果
- 归并(Merge): Map任务全部结束前进行, 得到大文件放在本地磁盘
- JobTracker监控Map执行并通知Reduce领取数据

Combine vs Merge: 两个 $\langle "a", 1 \rangle$ 和 $\langle "a", 1 \rangle$ 。合并(Combine)得 $\langle "a", 2 \rangle$; 归并(Merge)得 $\langle "a", \langle 1, 1 \rangle \rangle$

Shuffle过程——Reduce端:

- Reduce通过RPC询问JobTracker Map是否完成, 完成则领取数据
- 数据先放入缓存, 来自不同Map机器, 先归并再合并, 写入磁盘
- 多个溢写文件归并成排序后的大文件
- 数据很少时直接在缓存中归并输出

MapReduce应用: 关系代数运算(选择/投影/并/交/差/连接)、分组与聚合运算、矩阵-向量乘法、矩阵乘法

关系自然连接示例: $R(A,B)$ 和 $S(B,C)$ 自然连接

- Map: R 元组 $\langle a, b \rangle \rightarrow \langle b, \langle R, a \rangle \rangle$; S 元组 $\langle b, c \rangle \rightarrow \langle b, \langle S, c \rangle \rangle$
- Reduce: 相同B值的元组合并, 输出 $\langle a, b, c \rangle$

MapReduce 1.0的主要问题:

1. JobTracker是集中处理点, 存在单点故障
2. JobTracker任务过多, 资源消耗大
3. 当job非常多时内存开销极大 (Hadoop MapReduce上限约4000节点)
4. TaskTracker端以map/reduce task数目表示资源过于简单, 没考虑CPU/内存占用
5. 强制划分map slot和reduce slot, 只有一种task时造成资源浪费

MapReduce 2.0 / YARN设计改进：

- 1. 大大减小JobTracker(即ResourceManager)资源消耗，监测任务状态的程序分布式化
- 2. ApplicationMaster可变，用户可为不同编程模型写自己的AppMaster
- 3. 资源表示以内存为单位（目前不考虑CPU），比剩余slot更合理
- 4. 消除了map/reduce slot分开造成的资源闲置

3.4 负载均衡

背景：单服务器性能有天花板，需要设计高性能集群。复杂性体现在需要增加任务分配器（负载均衡器）和选择合适分配算法。

三种负载均衡器对比：

类型	代表方案	性能	优点	缺点
DNS	DNS解析	—	简单、成本低、就近访问	更新不及时、扩展性差、分配策略简单
硬件	F5、A10、Radware、Citrix	百万级并发(200-800万/秒)	功能全面、稳定、支持安全防护	价格昂贵(20-70万)、扩展差
软件	Nginx(7层)、LVS(4层)	Nginx万级(5万/秒)、LVS十万级(80万/秒)	简单、便宜、灵活可定制	性能一般、安全功能弱

七层负载均衡(OSI模型)：

- 二层：修改目标MAC地址
- 三层：通过IP转发
- 四层：修改IP+端口号(TCP/UDP协议层)
- 七层：根据URL、浏览器类别、语言等应用层内容进行负载

多级别部署：DNS(地理级别) → F5(集群级别) → Nginx(机器级别)

负载均衡算法：

算法	原理	优点	缺点
轮询(Round Robin)	按顺序轮流分配	简单	不感知服务器状态(CPU/核数差异)
加权轮询(Weighted RR)	按硬件配置静态权重分配	解决配置差异	不感知运行时状态
负载最低优先	分配给连接数/CPU/IO最低服务器	感知运行状态、较完美	复杂度大幅上升、时间间隔难调优
性能最优优先	分配给响应最快的服务器（客户端角度）	快速响应	收集统计消耗性能，工业用采样

算法	原理	优点	缺点
Hash类	源地址Hash/Session ID Hash/ID Hash	满足特定业务需求(如会话保持)	负载分布可能不均

Hash类算法详解：

- **源地址Hash：**同一源IP→同一服务器。适合事务/会话业务(如网上银行登录)
- **ID Hash：**临时数据ID(如session id)→同一服务器

实际使用观察：负载最低优先算法看起来完美，但实际应用不如轮询(含加权轮询)广泛，因为复杂度代价太高。

3.5 消息队列

理论基础：成功的分布式系统依赖于隐藏或简化消息传递的通信模型。分布在网络中的多进程通信，底层网络不可靠，没有基于共享内存的原语，基于低层级消息传递，比共享内存更难。消息队列隐藏了大多数复杂的信息传递。类似电子邮件的高级消息排队模型。

消息队列的九大分类维度：

1. **寻址类型：**直接寻址、队列寻址、主题寻址
2. **阻塞类型：**同步阻塞、非阻塞
3. **缓存类型：**瞬态(Transient)、持久(Persistent)
4. **内容类型：**结构化、非结构化
5. **确认类型：**自动确认、手动确认
6. **接收节点数：**点对点、多播、任播、地域性群播、广播
7. **通讯方向：**单向、双向半双工、双向全双工
8. **发起方：**拉取(Pull)、推送(Push)
9. **消息存储：**
 - **持续通信(Persistent)：**消息由中间件存储，收发方无需同步，接收器下次运行时得到消息
 - **瞬态通信(Transient)：**收发双方均运行才存储，中断或无法传递则丢弃

两种主要模式：消息排队(Message Queuing)、发布-订阅(Publish-Subscribe)

面向消息的持久通信(Message-Oriented Persistent Communication)：

- 持久异步通信
- 为消息提供中间形态存储容量
- 不需要收发方在传输期间保持活跃
- 支持允许消息传输的时间开销

消息队列的关键能力：异步交互、可靠服务交付、通过中间消息服务器处理消息、支持数据库集成

工作原理：应用程序在特定队列中插入消息 → 通过通信服务器转发 → 到达目的地队列。接收双方无需同步。发送者可保证消息最终被插入收件人队列但不能保证何时插入。

同步 vs 异步通信：

- 同步：双方需同时在线

- 异步持久：可补偿收发方速度偏差（不补偿速度差异），存在缓冲区溢出风险

队列(Queues)的作用：

- 作为收发之间的命名消息目的地
- 允许进程独立执行和失败
- 可掩盖进程失败和通信失败

消息优先级：最高优先级优先(Highest Priority First)、加权公平调度(Weighted Fair Scheduling)

MOM(面向消息的中间件)功能：支持多种消息传递模型、队列管理、连接管理、服务质量(QoS)、数据转换

队列管理器(Queue Manager)：

- 基本功能：创建/删除队列、启动/停止队列、修改队列属性、监视性能/故障/恢复
- 高级功能：重新格式化数据、全局翻译、消息代理(Message Broker)
- 可配置为将消息转发给其他队列管理器

队列的解耦能力：

- **时间解耦(Time Decoupling)：**收发方无需同时在线
- **位置解耦(Location Decoupling)：**互相无需知道物理位置

消息队列模式：排队消息模式(Queueing)、消息传递/点对点(Message Passing/P2P)、发布订阅(Pub-Sub)、请求回复+队列(Request/Reply with Queues)

3.6 分布式服务框架

通信基础：客户端提出Web Services请求(XML methodCall)，服务端传输反馈(XML methodResponse)，通过Service Stub(Stub)和HTTP协议完成。

三层架构（自下而上）：

RPC层：

- 底层通讯框架、序列化/反序列化框架
- Remoting框架屏蔽底层通信协议和序列化差异
- 支持：Binary Protocol、Text Protocol、Web Services、Restful、Hessian
- 底层基于NIO Framework和HTTP Based Protocol

Filter Chain层：

- 服务调用职责链
- 提供：负载均衡(LoadBalance)、过滤器链(Filter chain)、超时/重发(Timeout/resend)
- 还有：服务调用性能统计、完成通知机制、失败重发

Service层：

- 消费者端：Java动态代理(Dynamic Proxy, cglib/Javassist)，将服务接口封装成远程调用
- 服务提供者端：Java反射，根据接口名/方法名/参数列表反射调用本地实现
- 业务层：Spring Bean，平台负责将业务接口发布为远程服务

两大功能中心：

服务注册中心：

- 负责服务发布和通知
- 对等集群部署，单一宕机不影响整体；全部宕机只影响新注册不影响已发布服务
- 具体实现：HSF用ConfigServer(数据库)、Dubbo用Zookeeper、SpringCloud用Eureka

服务治理中心：

- 包含服务治理接口和服务质量Portal
- 目标：持续化服务、防止架构腐化、保证高质量运行
- 可视化分析和维护运行状态、历史数据、健康度、调用关系

服务订阅与发布：

- 配置化发布和引用(XML配置)，降低代码侵入
- 服务自动发现：注册中心推送地址，消费者不需配置，地址透明化
- 在线注册/去注册：运行态注册新服务或取消已有服务

服务路由：

- 默认路由策略：随机、轮循、权重（避免重复开发）
- 粘滞连接：总向同一提供方请求，宕机才切换
- 路由定制：用户自定义路由策略

集群容错：

策略	行为	适用场景
Failover	失败自动切换，重试其他服务器	读操作、幂等性写操作
Failback	后台记录失败请求，定时重发	消息通知操作
Failfast	只发起一次，失败立即报错	非幂等性写操作

服务调用方式：

- 同步调用：阻塞等待响应
- 异步调用：不阻塞立即返回，服务端应答后异步通知
- 并行调用：同时批量发起，集中等待

多协议支持：私有协议(二进制、可定制扩展)、公有协议(Web Service、外部对接)

序列化方式：二进制(Thrift、Protocol Buffer，高性能)、文本(JSON、XML，通用可读)

统一配置：本地静态配置(一次部署不修改)、配置中心动态配置(实时生效)

3.7 分布式幂等性

定义：同一操作的一次或多次请求结果一致，不会因多次点击产生副作用。

重要性：服务无状态本质导致对业务敏感性弱，不支持幂等会造成：超卖现象、重复转账、重复扣款/付款、重复增加金币/积分/优惠券。

产生场景：

1. 网络波动引起重复请求
2. 分布式消息消费
3. 用户无意/有意重复操作
4. 未关闭的重试机制 (Nginx重试、RPC重试、业务层重试)

数据库操作幂等性分析：

- 新增类：**不具备**幂等性
- 查询类：**天然幂等** (不产生新数据)
- 更新类：
 - 基于主键计算式Update (`SET price = price-1`)：**不具备**
 - 基于主键非计算式Update (`SET price = newPrice`)：**具备**
 - 基于条件查询更新：**一般不具备**
- 删除类：基于主键Delete **具备**；逻辑删除(update)也**具备**

五种控制方案：

1. **数据库加锁法**：关键资源串行化，引入效率问题和死锁问题
2. **全局唯一ID法**：操作前判断全局ID是否存在，实现困难，与业务解绑冲突
3. **去重表法**：利用唯一标识(如订单号)判断是否重复执行，有唯一标识时最优
4. **多版本控制法**：每次操作加版本号，需管理版本号并配合日志实现最终一致性
5. **状态机控制法**：适合状态流转场景(如订单创建=0，付款成功=100，付款失败=99)，通过状态值大小做幂等

3.8 微服务架构

定义：将业务逻辑拆分成一系列小而松散耦合的分布式组件，每个组件称为微服务。每个微服务执行单独任务或功能，可被其他微服务调用。

微服务六大特征：负责单个功能、单独部署、包含≥1个进程、拥有自己数据存储、小团队可维护多个、可替换

SOA vs 微服务：

- SOA：通过ESB通信，业务逻辑在中间层（路由、转换、组织）
- 微服务：降低ESB依赖，业务逻辑分布在各服务终端；基于HTTP/JSON标准协议；可选轻量级消息总线/网关

三种集成方式：

1. **点对点方式**：微服务直接调用REST API。简单时可行，复杂度提升后不可维护。尽量不采用。
2. **API网关方式（应用最广泛）**：
 - 所有客户端通过统一网关接入
 - 网关处理所有非业务功能
 - 优势：网关层次抽象（统一规范接口）、轻量消息路由/格式转换、统一安全/监控/限流、微服务更轻量只需关注业务逻辑
3. **消息代理方式**：通过队列和订阅主题异步发布订阅。生产者/消费者模式解耦。通过AMQP、MQTT等异步消息规范。

3.9 服务发现

服务注册表：服务发现核心，包含服务实例网络地址的数据库。需高可用且随时更新。若干服务端使用复制协议保持一致性。

客户端发现模式：

- 客户端查询服务注册表，负载均衡算法选择实例
- 服务实例启动时注册、终止时删除，心跳机制定期刷新
- 优点：相对直接，可智能负载均衡（如哈希一致性）
- 缺点：客户端与服务注册绑定，需为每个语言/框架实现发现逻辑

服务端发现模式：

- 客户端通过负载均衡器发请求，负载均衡器查询注册表并转发
- 案例：AWS ELB均衡VPC内外部流量；EC2/ECS注册在ELB而非独立注册表
- 优点：客户端无需关注发现细节，减少发现逻辑；部分环境免费提供
- 缺点：负载均衡器需配置和管理的高可用组件

自注册模式：服务实例自己注册/注销+心跳。简单但需在每个语言/框架内实现注册代码。

第三方注册模式：服务注册器自动跟踪实例变更并注册/注销。服务与注册表解耦，但需配置和管理高可用组件。

3.10 数据去中心化

- 单体架构：所有服务模块共享中心数据库
- 微服务：每个微服务有自己的私有数据库，只能访问自己的数据库
- 跨服务事务不能直接访问其他数据库，必须通过对微服务操作
- 优势：降低耦合度，不同服务可用不同数据库技术(SQL/NoSQL)
- 复杂场景通常在客户端或网关处理

3.11 Docker容器技术

Docker引擎：C/S架构应用。Server是常驻进程；REST API实现Client-Server交互；CLI实现容器和镜像管理。

Docker vs 虚拟机：Docker没有Guest OS和Hypervisor层，直接使用物理机硬件资源。CPU和内存利用率更高。

Docker四大优点：

1. 更快速的交付和部署（完美辅助持续开发流程）
2. 高效的部署和扩容（几乎任意平台运行，轻松动态负载管理，趋近实时的扩容/下线）
3. 更高的资源利用率（一台主机运行数千容器，基本不消耗额外资源。vs 虚拟机10应用需10个VM）
4. 更简单的管理（增量修改和更新，自动化高效管理）

Docker三大缺点：

1. 资源隔离不如虚拟机（cgroup限制最大值，不能隔绝其他程序占用）
2. 安全性问题（不分辨执行命令的用户，A可删除B创建的容器）
3. 版本兼容问题（快速更新，细节调整大，核心模块依赖高版本内核）

Docker C/S架构：Client通过接口与Server进程通信，Client和Server可同机或跨主机远程通信。

三大核心概念：

镜像(Image)：

- 只读模板（含完整OS环境+应用）
- 一个镜像可创建多个容器
- 技术实现：多层只读层(Read-Only Layer)重叠，Union File System统一视角。除了最下层每层指向下一层

仓库(Repository)：

- 集中存放镜像（类似Git）
- 分为公开(Public)和私有(Private)。最大公开仓库是Docker Hub
- push上传镜像，pull下载镜像
- 注册服务器类似GitHub托管服务

容器(Container)：

- 镜像的运行实例，可启动/开始/停止/删除
- 相互隔离的安全平台。简易版Linux环境+应用程序
- 与镜像的区别：最上层是**可读写层(Read-Write Layer)**
- 运行态容器 = 可读写统一文件系统 + 隔离进程空间

3.12 容器调度管理工具

三个主流工具：

Swarm(Docker Swarm)：

- Docker原生集群化工具，整合到Docker Toolbox
- 使用与Docker标准API一致的API，相同命令操作
- 由多个代理(agent/节点node)组成，节点启动Docker daemon时打开端口支持远程API
- 节点可贴标签(label)用于过滤集群
- **三种调度策略**：
 - random：随机选择（开发测试阶段）
 - spread(默认)：选占用资源最少节点，保证所有节点均匀使用
 - binpack：尽可能填满一个节点，保证更多空余节点

Apache Mesos & Mesosphere Marathon：

- 建立高效可扩展系统，支持多种框架
- Mesos提供底层轻量资源共享层，统一接口访问集群资源
- Mesos负责委派授权不负责调度
- Marathon提供：约束(constraints)、健康检查(health checks)、服务发现(service discovery)、负载均衡(load balancing)
- **四个模块**：ZooKeeper(查找Mesos master地址)、Marathon(启动监控扩展容器)、Mesos master(分配任务)、Mesos slave(运行容器+报告资源)

Kubernetes(K8s):

- Google开源, 自动部署、伸缩和管理容器化应用
 - **核心概念:**
 - **Pod:** 成组部署调度的容器集合(1-5个容器协作提供Service), 这是K8s与Swarm/Mesos的主要区别
 - 扁平网络模式: 同Pod容器共享IP、用localhost端口通讯
 - **Label:** 搜索和更新多个对象
 - DNS服务器: 监控新服务, 通过名字访问
 - **Replication Controller:** 实例化Pod, 管理监控容器
 - **调度器:** 寻找PodSpec.NodeName为空的pods调度。可用PodSpec.NodeName绕过调度器
 - **谓词(Predicates):** 强制规则。PodFitPorts(无端口冲突)、PodFitsResource(足够资源)、NoDiskConflict(足够空间)、MatchNodeSelector(匹配选择器)、HostName(匹配host)。无满足则pod挂起
 - **优先级(Priorities):** 有多个满足时用优先级打分。LeastRequestedPriority(最小CPU/内存百分比节点最优)、BalanceResourceAllocation(类似内存CPU使用)、ServicesSpreadingPriority(不同pods节点优先)、EqualPriority(测试用)
-

第4章 数据层的软件架构技术

4.1 数据的基本概念

数据定义:

- 对客观事件进行记录并可以鉴别的符号, 是对客观事物的性质、状态以及相互关系等进行记载的物理符号或这些物理符号的组合。它是可识别的、抽象的符号
- 表示形态: 数字、文字、字母、数字符号组合、图形、图像、视频、音频、属性/数量/位置抽象表示
- 计算机科学中: 所有能输入到计算机并被计算机程序处理的符号介质总称

数据与信息: 数据是信息的表达和载体, 信息是数据的内涵(形与质的关系)。数据本身没有意义, 只有对实体行为产生影响时才成为信息。

数据与语义: 数据的表现形式不能完全表达其内容, 需要经过解释。数据的解释是对数据含义的说明, 数据的含义称为数据的语义。数据与其语义不可分。

核心公式: 数据 + 语义 + 逻辑 = 业务; 代码 + 业务 = 软件应用系统

4.2 数据层面临的挑战

三大变化趋势:

1. Stand alone → Interoperability → Transboundary (单机到互操作到跨界)
2. Customized user data (定制化用户数据)
3. IoT data (物联网数据)

- 总体特征: 复杂、多用户、多片段、异构、海量、离散
- 核心挑战: 面向海量数据的高并发复杂分布式异构系统

三大核心问题:

1. 数据量过大，单机存储不下（MySQL 5.7单表极限5000万条，300万时已需优化）
2. 数据量过大，单机内存存不下数据索引（索引存于内存，数据越大→索引越大→内存消耗越多；每次数据加入需维持索引）
3. 访问量过大，读写混合情况下一个实例不能承受

演进历程：

- 文件缓存 → 多台Web服务器不能共享，小文件缓存带来高IO压力
- Memcached应运而生管理缓存
- 缓存原则：频繁被访问的数据放置于缓存中供频繁访问
- 垂直分离：按业务划分将不同数据放在不同数据库
- 主从复制：容灾备份，缓存备份，保证数据完整性
- 读写分离：增删改是写，查为读；读从查询服务器读，写向写服务器写；结合缓存实现性能大幅提升

数据量不可控的原因：未分库分表时表和数据随时间业务发展越来越多，增删改查开销越来越大。无法分布式部署，单台服务器资源有限，最终遭遇瓶颈。

云时代对DB的新需求：

1. **高性能：**高并发读写（每秒上万次），硬盘IO无法承受
2. **海量存储：**对海量数据SQL查询效率极其低下
3. **高伸缩性与高可用性：**无法简单添加节点扩展，需停机维护和数据迁移

三大技术方向：

- 存储高性能：读写分离、数据缓存、分库分表、NoSQL
- 存储高可用：主从、CAP理论
- 存储高扩展：分库分表、NoSQL

4.3 读写分离

基本架构：数据库服务器搭建主从集群（一主一从/一主多从）。主机负责读写操作，从机只负责读操作。主机通过复制同步数据到从机。每台服务器存储所有业务数据。业务服务器将写操作发给主机，读操作发给从机。

主从复制延迟问题：写入后立刻读取，此时访问从机可能主机尚未复制数据，读不到最新数据。

三种解决方案：

1. 写操作后的读操作指定发给主机（如注册后登录时也读主机）
2. 读从机失败后再读一次主机
3. 关键业务读写全部指向主机，非关键业务采用读写分离

实现方式：中间件封装、程序代码封装

4.4 主备复制(Master-Backup)

原理：

1. 主机存储数据，通过复制通道复制到备机
2. 正常时客户端读写都发给主机，备机不对外提供服务

3. 主机故障时系统不可用，不能读写，但数据未全部丢失
4. 主机可恢复则继续；不可恢复需人工升级备机为主机、增加新备机
5. 写入主机但未复制到备机的数据会丢失
6. 备机只起备份作用；复制延迟不影响业务但主机宕机时可能丢失较多数据

优点：客户端不需要感知备机存在；主机备机只需数据复制无需复杂倒换

缺点：备机仅备份不提供服务(硬件浪费)；故障需人工干预无法自动恢复

4.5 主从复制(Master-Slave)

与主备的区别：正常情况下读操作也可发给从机，从机参与对外服务。

原理：

1. 主机存储数据，复制到从机
2. 正常时写发送给主机，读可发送给主机或从机（随机读/轮询读/只读主机）
3. 主机故障时写操作不可用但读操作不受影响
4. 主机恢复则继续；不可恢复需人工升级从机为主机
5. 写入主机未复制到从机的数据丢失
6. 主从间延迟导致主从读取数据不一致

优点（相对主备）：主机故障读操作不受影响；从机提供读操作发挥硬件性能

缺点：比主备复杂（客户端需感知主从关系）；同样需人工处理故障

4.6 主主复制(Master-Master)

原理：两台主机互为主机，互相复制数据，客户端任意挑选一台读写。

- 正常时客户端任意读写
- 一台故障时客户端只访问另一台
- 故障主机恢复则继续互访；不能恢复需人工增加新机器

关键问题：两台主机间复制延迟 → 刚写入的数据在另一台读不到。双写ID冲突（两个库新插数据ID均为100）。购买数量冲突（甲从A买1件乙从B买2件，复制B→A则两库都是98而非97）。

优点：无倒换概念；客户端无需区分主备身份

致命问题：很多数据无法双向复制

4.7 主备/主从倒换设计

共性核心问题：

- 主机故障后无法写操作
- 主机无法恢复需人工指定新主机

关键设计点：状态判断(传递渠道+检测内容)、倒换决策(时机+策略+自动程度)、数据冲突

三种状态传递方式：

互连式：

- 主备机直接建立状态传递通道（网络/串口，单向/双向，单通道/多通道/混合通道）
- 可用虚拟IP或客户端记录双方IP
- 缺点：通道本身故障导致备机误升为主机；多通道时状态决策更复杂

中介式：

- 引入第三方中介，主机备机都连接中介传递状态
- 状态决策5步骤：初始都是备机；断连即降级；主机断连时备机被升级；网络中断恢复后旧主机以新备机身份上报；正常连接按实际状态决策
- 优点：连接管理更简单、状态决策更简单

模拟式：

- 备机模拟客户端向主机发起模拟读写，根据响应判断状态
- 优点：实现更简单，省去状态传递通道
- 缺点：只有响应信息(404/超时/响应时间>3s)，可能出现偏差

4.8 数据集群

数据集中集群（一主多备/从）：

- 数据只能写主机，读灵活配置
- 复杂度高：多通道增加主机复制压力；多通道情况不一导致数据不一致（需检查和修正）；多备对单主状态检测结果不一致；主机宕机选主需算法
- 适合：数据量不大、集群机器不多（如ZooKeeper，个位数机器）

数据分散集群：

- 每台服务器负责部分数据 + 备份部分数据
- 核心问题：数据分区算法
- 均衡性（基本均衡）、容错性（故障后重分配）、可伸缩性（扩容后自动迁移并保持均衡）
- 需角色执行分配算法：独立服务器(如HDFS)或选举主机(如Elasticsearch)
- 写数据：客户端可向任意服务器读写
- 适合：海量数据、集群庞大（如Hadoop，上千台）

4.9 分库分表

本质：数据拆分，分而治之。将一个表结构分为多个表或数据分片，可放同一库/不同库/不同实例。

基础概念：

- 数据库：文件的集合，依某种数据模型组织存放于二级存储器
- 数据库实例：程序，位于用户和操作系统之间的数据管理软件，通过实例和数据库交互
- 数据库由二进制文件组成，需实例完成SELECT/UPDATE/INSERT/DELETE操作

两大拆分类型：

垂直拆分：按业务维度拆分，每个库(表)与原有结构不同。

- 按业务分库(User/Pay/Commodity)

- 按业务分表
- 冷热分离：冷数据(变化更新频率低、查询次数多)和热数据(变化更新频率高、活跃)
- 案例：微博系统中文章属性(冷数据)用MyISAM引擎；浏览/回复/点赞数(热数据)用InnoDB引擎
- 优点：业务清晰、易扩展、便于管理、动静分离/冷热分离、维护简单
- 缺点：跨表Join困难(只能接口方式)、单库性能瓶颈、事务处理复杂

水平拆分：按分片算法拆分，每个库(表)结构相同。按行切分分配。

- 案例：100万用户/请求，分100个表+10个数据库：每表1万请求、每库10万请求
- 优点：单表数据量级可控、性能提高、应用层改造少、稳定性负载提高
- 缺点：跨库Join性能差、拆分规则难抽象、分片事务一致性难解决、数据扩容维护量大

共通问题：分布式事务、跨节点Join、跨节点合并排序/分页、多数据源管理

总结：垂直切分偏向业务拆分过程；水平切分偏向技术性能指标。

操作时机：表数量达量级；数据库吞吐量达瓶颈；扩容时希望配置改变最少

发展阶段：单库单表 → 单库多表 → 多库多表

典型实例：16亿条用户消费记录。MySQL单表极限5000万条。分解到4个实例×2个数据库×4个表=32个表。

路由与分片维度：

- **按哈希切片：**对某字段求哈希÷分片总数取模。数据均匀分散效果好，但查询需聚合
- **按时间切片：**按时间范围分布数据

三种解决方案：

客户端分片：应用层直接操作分片逻辑。分片规则在多个节点间同步，每层嵌入切片逻辑。

- 应用层直接实现：简单快速但侵入业务，连接数多，服务器池维护压力大
- 定制JDBC协议：开发者专注业务逻辑，Sharding JDBC等流行框架
- 定制ORM框架：分片规则实现到ORM(如Mybatis)

代理分片：应用层和数据库层间增加代理层(如Cobar、Mycat)。开发人员只需关心业务逻辑。缺点：增加网络传输影响性能、需维护代理层增加成本。

分布式数据库(OceanBase/TiDB等)：内部实现伸缩和分布式事务，对使用者透明。TiDB提供JDBC接口。目前不太适用于交易系统，较多用于大数据日志/统计/查询/社交网络。

分片规则变更处理（五步骤）：

1. 新旧分片规则双写
2. 历史数据按新规则迁移写入
3. 查询规则切换到新规则
4. 下线双写逻辑
5. 删除旧规则历史数据

数据一致性问题：先清理旧数据→迁移→全量对比→迭代清洗→抽样对比(数据量巨大时)→记录更新操作日志→迁移后根据日志+历史数据共同决定最新状态(最终一致性)

查询问题三种方案：

1. 多分片表查询后合并数据集（效率很低）
2. 按查询需求定义多分片维度形成多张分片表（空间换时间）
3. 通过搜索引擎解决，有实时要求还需实时搜索（难度大）

分布式事务核心原则：尽量把同一组数据放到同一台数据库服务器上，利用本地事务强一致性，使数据实现自治。

4.10 分库分表中间件详解

Mycat:

- 数据库中间件：读写分离、分表分库、容灾备份、多租户应用、云平台基础设施
- 不存储数据，数据在MySQL。像MySQL的存储引擎
- **SQL处理流程：**拦截SQL → 解析 → 分片分析 → 路由分析 → 读写分离分析 → 缓存分析 → 发往后端真实DB → 返回结果适当处理 → 返回客户端
- 获取分片字段值 → 匹配分片函数 → 得到分片列表 → SQL发往分片执行 → 收集处理结果输出
- **典型场景：**读写分离主从切换(最简单)；1000万+表分片(最大支持1000亿单表)；多租户(不改造程序)；报表系统；替代Hbase分析大数据；100亿条记录3秒内查询(主键/范围/属性查询)

Sharding-JDBC:

- 当当开源，轻量Java框架，水平分片框架
- 增强版JDBC驱动，旧代码迁移成本几乎为零
- 直接封装JDBC API，客户端直连数据库，jar包提供服务
- 无proxy代理层，无需额外部署和依赖，DBA不改变运维方式
- 适用于任何基于Java的ORM框架(JPA/Hibernate/Mybatis/Spring JDBC Template等)
- 可基于任何第三方连接池(DBCP/C3P0/BoneCP/Druid等)
- 分片灵活：支持等号、between、in多维度分片，支持多分片键
- SQL功能完善：join、aggregation(含avg)、order by、group by、limit、or查询、Binding Table、笛卡尔积查询

分片规则配置：支持分片策略自定义、复数分片键、多运算符分片。如：根据用户ID分库+根据订单ID分表（分库分表结合）；根据年分库+月份+用户区域ID分表（多片键）

JDBC规范重写：封装DataSource/Connection/Statement/PreparedStatement/ResultSet五个核心接口。最大实现JDBC协议(含addBatch等)。未实现：Connection游标、存储过程、savePoint、ResultSet向前遍历和修改、JDBC 4.1+

SQL解析：使用Druid作为SQL解析器（速度是其他解析器的几十倍）。支持复杂SQL(join/aggregation/order by/group by/limit/or查询)。暂不支持union、部分子查询、函数内分片。

SQL改写：逻辑表名→真实表名。替换分片环境不正确功能。例：avg不能简单avg1+avg2+avg3/3，需改写为(sum1+sum2+sum3)/(count1+count2+count3)。

SQL路由（三种）：单表路由(可能落入多库因支持between/in)、Binding表路由(分库分表规则一致的主从表)、笛卡尔积路由(最复杂，非Binding表关联查询=笛卡尔积组合，性能低，连接数高)

SQL执行：多线程并发执行，完成addBatch等批量处理。

结果归并（四类）：

1. 普通遍历类：按顺序遍历ResultSet集合
2. 排序类：归并排序算法整合
3. 聚合类：比较型(max/min取最大值)、累加型(sum/count累加)、平均值型(通过改写后的sum/count计算)
4. 分组类：最复杂，所有ResultSet放入内存→map-reduce算法分组→排序和聚合处理。最耗内存和性能。可用limit合理限制

4.11 数据缓存

缓存定义：用于存储数据的硬件或软件组件，使后续更快访问响应数据。缓存中数据可能是提前计算好的结果、数据副本。

缓存作用的三大原理：写入更快存储；缓存到离应用最近位置；缓存到离用户最近位置。

缓存存储介质分类：

1. 内存：最快但无持久化（应用宕机数据难复原）
2. 硬盘：内存满或异常时持久化释放空间/备份
3. 数据库：Key-Value结构特殊数据库(如BerkeleyDB、Redis)，响应速度和吞吐量远高于关系型DB

四种缓存类型：

类型	说明	代表	应用场景
本地缓存	应用进程内，请求极快，无网络开销； 缺点是应用无法共享	Ehcache、Guava Cache	字典等常用数据
分布式缓存	独立应用，与应用隔离，多应用可共享	Redis、Memcached	复杂运算数据、热点数据
反向代理缓存	处理所有Web请求，有缓冲则直接返回	Varnish、Nginx、Squid	小体积静态文件(css/js/图片)
CDN缓存	内容发布到最近用户网络边缘	CDN服务商	解决带宽小/访问量大/网点分布不均问题

CDN目标：解决因网络带宽小/用户访问量大/网点分布不均造成的访问速度慢。关键技术：分布式存储、负载均衡、网络请求重定向、内容管理。

缓存核心指标：

- **命中率** = 返回正确结果数 / 请求缓存次数。衡量缓存有效性最重要指标。中等Web缓存40%命中率即为合理。
- **最大元素(最大空间)：**超过则触发清空策略。合理设置可提高命中率。

HTTP再验证(Revalidation)：检查缓存副本是否是服务器最新副本("新鲜度检测")。HTTP定义特殊请求快速检测。最常用If-Modified-Since首部。

- 服务器收到请求3种情况：
 1. 再验证命中/缓慢命中：对象未修改 → HTTP 304 Not Modified
 2. 再验证未命中：副本不同 → HTTP 200 OK(完整内容)
 3. 对象被删除 → HTTP 404 Not Found，缓存删除副本

缓存清空策略：

策略	原理	适用场景
FIFO	清除最先进入的数据	数据实效性要求
LFU	清除使用次数最少的元素	保证高频数据有效
LRU	清除最远时间戳的元素	热点数据保证

其他：过期时间最长/最近过期优先清理、随机清理、按关键字/内容长短清理等。

三种缓存读写模式：

Cache Aside：

- 失效：先查cache→未得→查DB→成功后放缓存
- 命中：从cache取到→返回
- 更新：先存DB→成功后→让缓存失效

Read/Write Through Pattern：

- Read Through：缓存失效时缓存服务自己加载数据，对应用透明
- Write Through：数据更新时如果命中缓存则更新缓存再由Cache同步更新DB；未命中直接更新DB返回

Write Behind Caching Pattern(write back)：

- 更新时只更新缓存不更新DB，缓存异步批量更新DB
- 优点：I/O极快，异步可合并同一数据多次操作，性能提高相当可观
- 致命缺点：非强一致性，极可能丢失数据(如Redis持久化无法绝对保证不丢失)

缓存扩容示例（2片→4片）：

1. 双写：同时对旧集群(Hash(key)%2)和新集群(Hash(key)%4)写
2. 迁移历史数据
3. 切读：读操作切换到新集群
4. 下线双写
5. 完成扩容

一致性哈希(Consistent Hashing)：

- 将哈希值空间组织成虚拟圆环(0~2^32-1)，0和2^32-1在零点重合
- 服务器通过ip/主机名哈希定位到环上
- 数据key哈希定位到环上，顺时针行走遇到的第一台服务器即为目标
- 容错性：服务器宕机只影响该服务器到环空间前一台服务器间的数据
- 可扩展性：增加服务器只影响新服务器到前一台服务器间的数据
- 虚拟节点机制：解决节点太少时分布不均匀的数据倾斜问题。对每个服务节点计算多个哈希作为虚拟节点。

缓存扩容的停止应用方案：停机→迁移历史数据→更改数据源配置→重启。简单高效避免不一致，适用晚上访问小或非核心服务。

缓存五大问题与解决方案：

1. 数据一致性：

- 先写缓存再写DB：缓存写成功DB写失败→脏读。**解决方案：改为先写DB再写缓存**
- 先写DB再写缓存：DB写成功缓存写失败→读不到数据。解决方案：缓存写失败回滚DB(增加复杂度)；或读缓存失败先读DB再回写缓存
- 缓存异步刷新：DB和缓存不在同一操作步骤。解决方案：按日志中刷新时间间隔和可能不一致时间进行同步

2. 缓存穿透：不存在的key大量高并发查询→每次都到DB→DB压力过大甚至崩溃。

- 解决方案：空值缓存(再次同样查询直接返回空)；恶意攻击时对查询条件设规则直接拒绝

3. 缓存并发：缓存key过期+高并发→多个请求同时发现过期→同时访问DB+回写→负载增加甚至DB崩溃。

- 分布式锁：每个key同时只有一个线程查后端
- 本地锁：只限一个服务节点。多节点时不完全解决
- 软过期：业务层存储过期时间，即将过期时延长时效+派遣线程更新+其他线程用旧数据等待

4. 缓存雪崩：缓存服务器重启或大量缓存同时失效→瞬时DB负载升高甚至崩溃。

- 更新锁机制：对缓存更新操作加锁
- 失效时间分片：不同数据不同失效时间。如基础10秒+随机0~2秒=10~12秒
- 后台更新机制：后台线程定时+频繁读取缓存；业务线程发现失效后通过MQ通知后台更新(适合缓存预热)
- 缓存集群：做缓存主从与水平分片

5. 缓存热点：热点数据高并发访问同一缓存→缓存服务器压力过大。

- 解决方案：复制多份缓存副本分散请求到多个缓存服务器

4.12 本地缓存实现方法

- 成员变量或局部变量实现：map结构缓存，仅限于类自身作用域，类间无法共享
- 静态变量实现
- **Ehcache**：最流行的纯Java开源缓存框架，轻量级，Hibernate集成。核心：cache manager(允许多实例)、cache(存放数据实质，实现Ehcache接口)。可轻松隔离多个缓存实例服务于不同业务场景。

4.13 分布式缓存数据分片扩展示例

商品信息服务(商品类目、商品城市、商品门店、商品优惠券)从2分片扩展到4分片：

双写→迁移历史数据→切读→下线双写

第5章 表示层的设计基础

5.1 设计目的

合理布局和设计版面，高效地传递视觉信息，充分吸引用户。

5.2 视觉习惯

三条基本规律：

1. 人眼的视线沿**水平方向**运动比沿垂直方向运动快且不易疲劳
2. 视线的变化习惯于**从左到右，从上到下和顺时针方向**运动
3. 斜线运动时：水平方向上下约45°范围内从左到右；垂直方向左右约45°范围内从上到下

页面视域：上部比下部注目程度高；左侧比右侧注目程度高。

最佳视域：(1)左上部 (2)中上部

视域部位心理感受：

- **上部：**轻快、上升、积极、愉悦
- **下部：**沉重、稳定、压抑、消沉
- **左侧：**轻松、舒展、自由、主动
- **右侧：**庄重、局限、拘谨、被动

设计建议：

- 滚动图片最好从右向左
- 线状布局应水平方向宽些
- 导航栏最好放在左侧或上部
- 要重点突出的内容最好放在左上部

页面规范：

- 固定页宽，一般不限定长度但不宜过长。一般应控制在1-3屏，最多不超过5屏。多屏信息可用超链接
- 文字疲劳场景：字号过小/行间距过密、高亮度文字/背景极暗
- 挥之不去的跟踪动效令人生厌

5.3 主从关系

核心思想：页面中构成元素不能同等对待，必须以点带面，让焦点先抓住浏览者，引发继续浏览的愿望，再展开来。

强化主从关系的五种方式：

1. 主从对象的大小对比
2. 弱化主体对象周围的内容
3. 主从对象的色彩对比（如桔红主体与绿色背景）
4. 重复主体对象：以不同角度或大小重复出现，提高与视线接触频率
5. 加强主从对象在形态上的对比

设计案例要点：主角突出但整体页面均衡；放大文字透视变形+黑暗背景对比；产品暗色调出现在亮色调中；主体为彩色（从非彩色背景中突出）

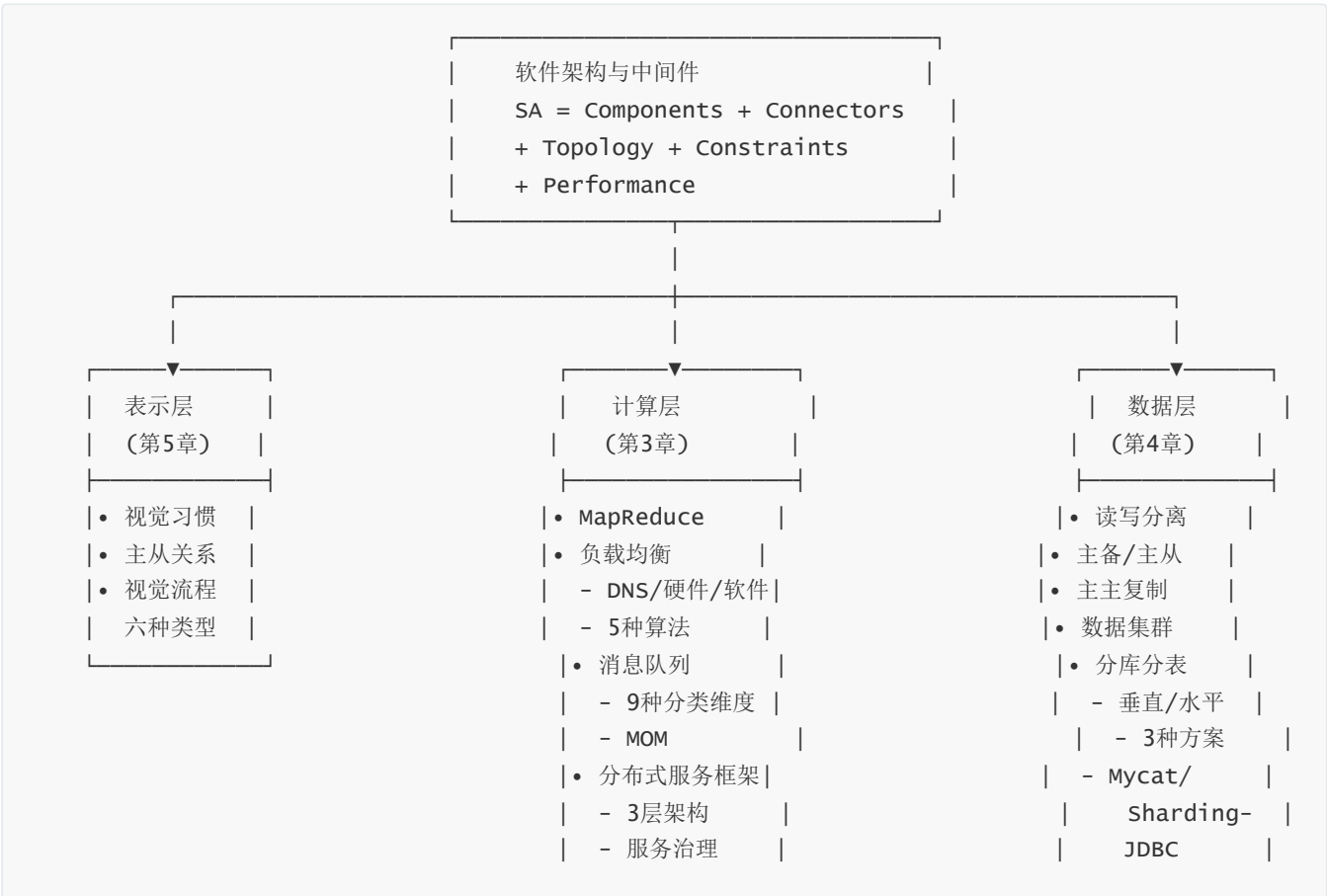
5.4 视觉流程

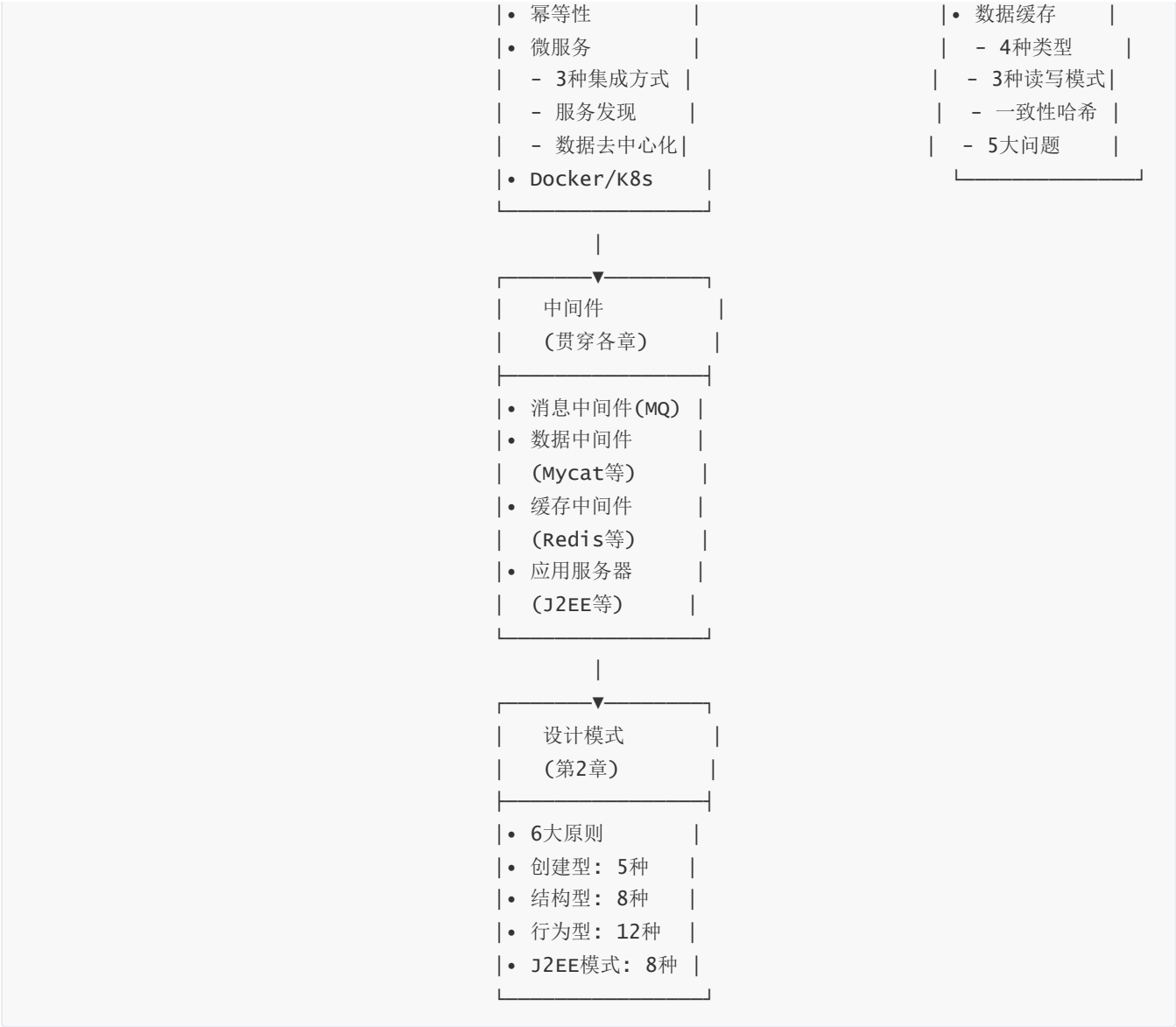
定义：视线随页面各构成元素在空间上沿一定轨迹运动的过程。视线运动是有规律的。

六种视觉流程：

类型	特点	子类型
单向视觉流程	单一方向顺序运动；由大到小、由主到次、由左到右、由上到下	横向、纵向、斜向
曲线视觉流程	沿曲线运动变化，更具节奏、韵律和曲线美	顺时针弧线、逆时针弧线
导向视觉流程	通过诱导元素主动引导视线，将元素依序串连	线形导向、形象导向、指示导向
重心视觉流程	以视觉心理重心(非几何中心)为起点；视线由重心移动到文字	—
反复视觉流程	相同/相似视觉要素规律性、富于秩序和节奏的逐次运动	—
散点视觉流程	无固定视线路径，自由分散但具有结构性和统一性	—

核心知识体系框架图





课程核心理念：面对大规模复杂软件系统，架构设计比算法和数据结构的选择更为重要。架构师需要在性能、安全、可用性、可修改性等质量属性间做出权衡，通过合理的设计模式和中间件技术，构建高可用、高性能、可扩展的分布式系统。正如Dijkstra在1968年指出的：**"The larger the project, the more essential the structuring!"**